

Universidad de Oriente	
Machine Learning	Clase 11 - Ejemplo 2 - Regresión de Ridge, Lasso y Red Elástica

```
In [23]: import pandas as pd
import os
import tarfile
import urllib.request
import numpy as np
from pandas.core.common import flatten
from plotnine import *
from array import *
import scipy.stats as stats
import math
import matplotlib as mpl
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn import linear_model
from sklearn.metrics import mean_squared_error, r2_score
import statsmodels.formula.api as smf
import statsmodels.stats.api as sms
```

Ejercicio 2

Este conjunto de datos explora la relación entre el uso de redes sociales y los niveles de productividad en los individuos. Incluye características de comportamiento, estilo de vida y actividad digital para simular patrones de uso del mundo real.

Variable	Descripción
age	Edad del individuo
daily_screen_time	Tiempo total de pantalla por día (horas)
social_media_hours	Tiempo dedicado a las redes sociales (horas)
study_hours	Tiempo productivo de trabajo o estudio
sleep_hours	Duración diaria del sueño
notifications_per_day	Número de notificaciones recibidas por día
focus_score	Nivel de concentración calculado (0–100)
addiction_level	Nivel de adicción: Bajo / Medio / Alto
productivity_score	Variable objetivo (0–100)

Parte 1

Realizar lo siguiente:

- Describir el conjunto de datos proporcionado (Notar que hay valores nulos en las variables incluso en la variable objetivo).
- Eliminar las observaciones que tienen valores faltantes y realizar la estandarización de los valores.
- Dividir el conjunto de datos en 25% de observaciones para la validación y el 75% restante para el entrenamiento
- Realizar un modelo de regresión Ridge del nivel de productividad con $\alpha=10$ y $\alpha=100$
- Comparar los dos modelos anterior con la regresión lineal múltiple.
- Realizar la validación cruzada para determinar el valor de α que proporciona un menor error cuadrático

In [25]: `data = pd.read_csv("social_media_productivity_6000.csv")`

In [27]: `data`

Out[27]:

	age	daily_screen_time	social_media_hours	study_hours	sleep_hours	notification
0	21.0	5.95	2.81	2.61	6.99	
1	34.0	3.82	2.33	0.28	7.47	
2	29.0	3.57	1.64	5.21	6.34	
3	25.0	10.27	4.37	4.28	4.49	
4	22.0	2.42	1.60	3.67	6.34	
...	...	...	...	...	...	...
5995	20.0	2.05	0.71	7.92	4.13	
5996	28.0	6.28	4.90	4.07	5.64	
5997	19.0	3.97	3.40	0.93	6.10	
5998	30.0	11.11	9.16	3.76	6.43	
5999	29.0	3.75	1.89	1.46	8.65	

6000 rows × 9 columns



In [29]: `#Mostrar la información de las variables`
`data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6000 entries, 0 to 5999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                    5880 non-null   float64
1   daily_screen_time      5880 non-null   float64
2   social_media_hours     5880 non-null   float64
3   study_hours            5880 non-null   float64
4   sleep_hours            5880 non-null   float64
5   notifications_per_day  5880 non-null   float64
6   focus_score            5880 non-null   float64
7   addiction_level        5880 non-null   object
8   productivity_score     5880 non-null   float64
dtypes: float64(8), object(1)
memory usage: 422.0+ KB
```

```
In [31]: #Contar los valores nulos
data.isnull().sum()
```

```
Out[31]: age                120
daily_screen_time          120
social_media_hours         120
study_hours                120
sleep_hours                120
notifications_per_day      120
focus_score               120
addiction_level            120
productivity_score         120
dtype: int64
```

```
In [33]: #Se eliminan los datos faltantes
data = data.dropna(axis=0)
#data.isnull().sum()
```

```
In [35]: #Separar variables independientes y la dependiente
predictoras = data.drop(["productivity_score", "addiction_level"], axis=1)
respuesta = data["productivity_score"]
```

```
In [37]: #Estandarizar las variables independientes

from sklearn.preprocessing import StandardScaler
escalar = StandardScaler()
escalar.fit(predictoras)

predictoras_esc = pd.DataFrame(escalar.transform(predictoras), columns = predict
predictoras_esc
```

Out[37]:

	age	daily_screen_time	social_media_hours	study_hours	sleep_hours	notifi
0	-0.856149	-0.341869	-0.619730	-0.634626	0.334203	
1	0.934427	-1.086400	-0.842160	-1.641953	0.668768	
2	0.245744	-1.173786	-1.161905	0.489428	-0.118853	
3	-0.305203	1.168166	0.103170	0.087363	-1.408320	
4	-0.718413	-1.575763	-1.180440	-0.176358	-0.118853	
...	...	...	...	...	...	...
4994	-0.029730	0.699775	0.719489	0.039806	-0.223404	
4995	-0.993886	-1.705094	-1.592864	1.661039	-1.659243	
4996	0.108007	-0.226519	0.348771	-0.003426	-0.606759	
4997	-1.131623	-1.033968	-0.346325	-1.360939	-0.286135	
4998	0.383480	1.461783	2.322843	-0.137448	-0.056122	

4999 rows × 7 columns



In [39]:

```
#Realizar la división de Los datos
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(predictoras_esc, respuesta,
```

In [41]:

```
#Importar clase
from sklearn.linear_model import Ridge

#Ajustar el modelo ridge con alfa=10
ridge_reg = Ridge(alpha = 10, solver = "auto")
ridge_reg.fit(X_train, y_train)

#Obtener coeficientes del modelo
#Intercepto
print("El intercepto que se tiene es %s" %ridge_reg.intercept_)
#Coeficientes de regresion
coefs_ridge_a = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":
coefs_ridge_a
```

El intercepto que se tiene es 37.662542554719565

Out[41]:

	Variables	Coeficientes
0	age	0.187078
1	daily_screen_time	-0.175359
2	social_media_hours	-16.945662
3	study_hours	22.263683
4	sleep_hours	7.530636
5	notifications_per_day	-2.694271
6	focus_score	-5.090162

In [42]:

```
#Ajustar el modelo ridge con alfa=100
ridge_reg_b = Ridge(alpha = 100, solver = "auto")
ridge_reg_b.fit(X_train, y_train)

#Obtener coeficientes del modelo
#Intercepto
print("El intercepto que se tiene es %s" %ridge_reg_b.intercept_)
#Coeficientes de regresion
coefs_ridge_b = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":
coefs_ridge_b
```

El intercepto que se tiene es 37.62873299438776

Out[42]:

	Variables	Coeficientes
0	age	0.026172
1	daily_screen_time	-2.189661
2	social_media_hours	-11.977202
3	study_hours	19.093606
4	sleep_hours	6.113143
5	notifications_per_day	-1.778178
6	focus_score	-1.114687

In [59]:

```
#Importar clase para la regresión lineal
from sklearn.linear_model import LinearRegression

#Ajustar el modelo
lm = LinearRegression()
lm.fit(X_train, y_train)

#Obtener error para cada uno
y_pred_lm = lm.predict(X_test)
y_pred_ridge_a = ridge_reg.predict(X_test)
y_pred_ridge_b = ridge_reg_b.predict(X_test)

mse_lm = mean_squared_error(y_test, y_pred_lm)
mse_ridge_a = mean_squared_error(y_test, y_pred_ridge_a)
mse_ridge_b = mean_squared_error(y_test, y_pred_ridge_b)
errores_ridge = [mse_lm, mse_ridge_a, mse_ridge_b,]
```

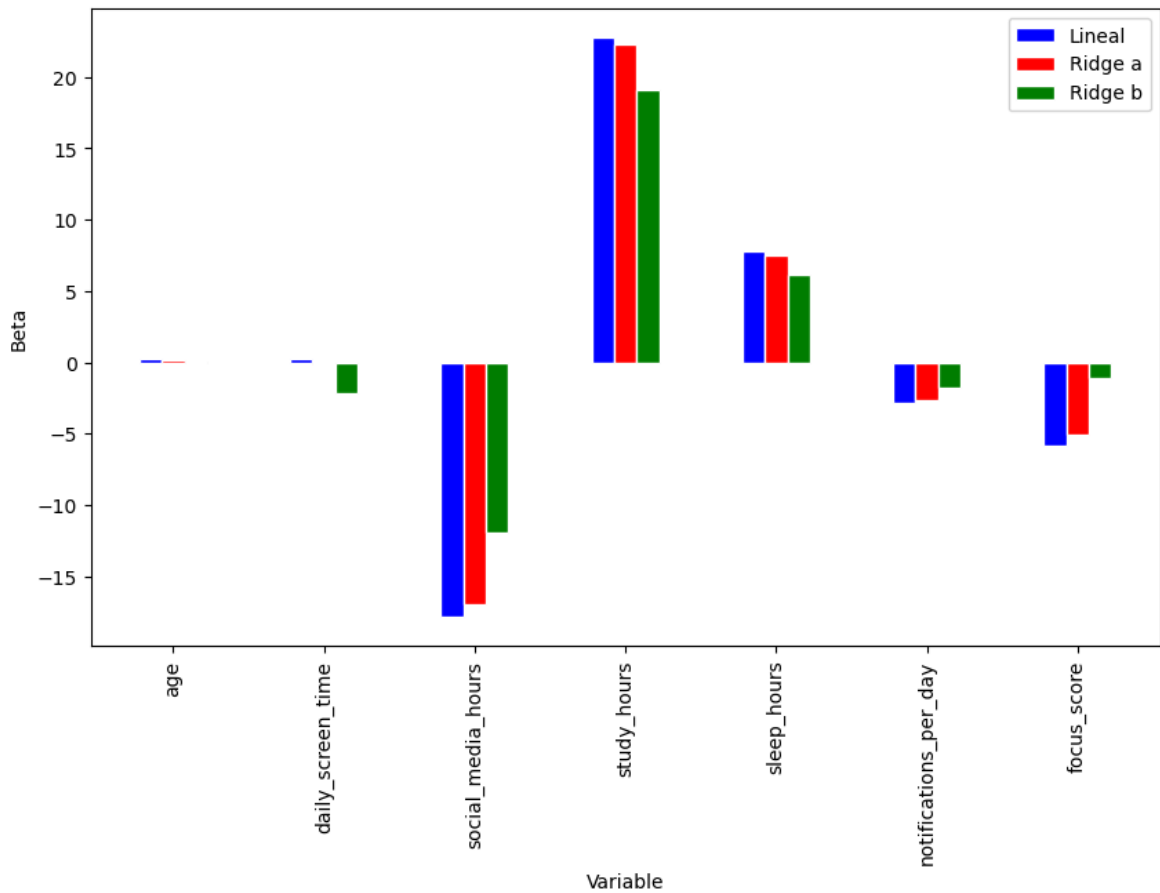
```
comparación_ridge = pd.DataFrame({"Modelo": ["Lineal", r"Ridge $\alpha=10$", r"Ridge $\alpha=100$"],
                                  "MSE": errores_ridge})

comparación_ridge
```

Out[59]:

	Modelo	MSE
0	Lineal	96.709343
1	Ridge $\alpha=10$	96.539473
2	Ridge $\alpha=100$	108.608601

In [68]: `fig = plt.figure(figsize=(10, 6))`*#Ancho de barra*`barWidth = 0.15`*#Definir posicion barras series*`r1 = np.arange(len(lm.coef_))``r2 = [x + barWidth for x in r1]``r3 = [x + barWidth for x in r2]`*#Pintar las barras*`plt.bar(r1, lm.coef_, color = "blue", width = barWidth, edgecolor = "white", label = "Lineal")``plt.bar(r2, ridge_reg.coef_, color = "red", width = barWidth, edgecolor = "white", label = "Ridge  $\alpha=10$ ")``plt.bar(r3, ridge_reg_b.coef_, color = "green", width = barWidth, edgecolor = "white", label = "Ridge $\alpha=100$ ")``plt.xticks([r + barWidth for r in range(len(lm.coef_))],
 [predictoras.columns[i] for i in range(len(predictoras.columns))], rotate = 45)``plt.legend()``plt.xlabel("Variable");``plt.ylabel("Beta");``plt.show(fig)`



In [124...

```

from sklearn.linear_model import RidgeCV
from sklearn.model_selection import KFold, cross_val_score

alphas = np.logspace(-3, 4, 100)
ridge = RidgeCV(alphas=alphas, cv=10)
ridge.fit(X_train, y_train)
kf = KFold(n_splits=10, shuffle=True, random_state=2026)

mse_mean = list()
mse_std = list()

for alpha in alphas:
    modelo_tmp = RidgeCV(alphas=[alpha], cv=None)
    scores = cross_val_score(modelo_tmp, X_train, y_train, scoring='neg_mean_squ

    mse = -scores
    mse_mean.append(mse.mean())
    mse_std.append(mse.std())

mse_mean = np.array(mse_mean)
mse_std = np.array(mse_std)

alpha_min_ridge = ridge.alpha_

idx_min = np.argmin(mse_mean)
mse_min = mse_mean[idx_min]
se_min = mse_std[idx_min]

idx_1se = np.where(mse_mean <= mse_min + se_min)[0][0]
alpha_1se = alphas[idx_1se]

```

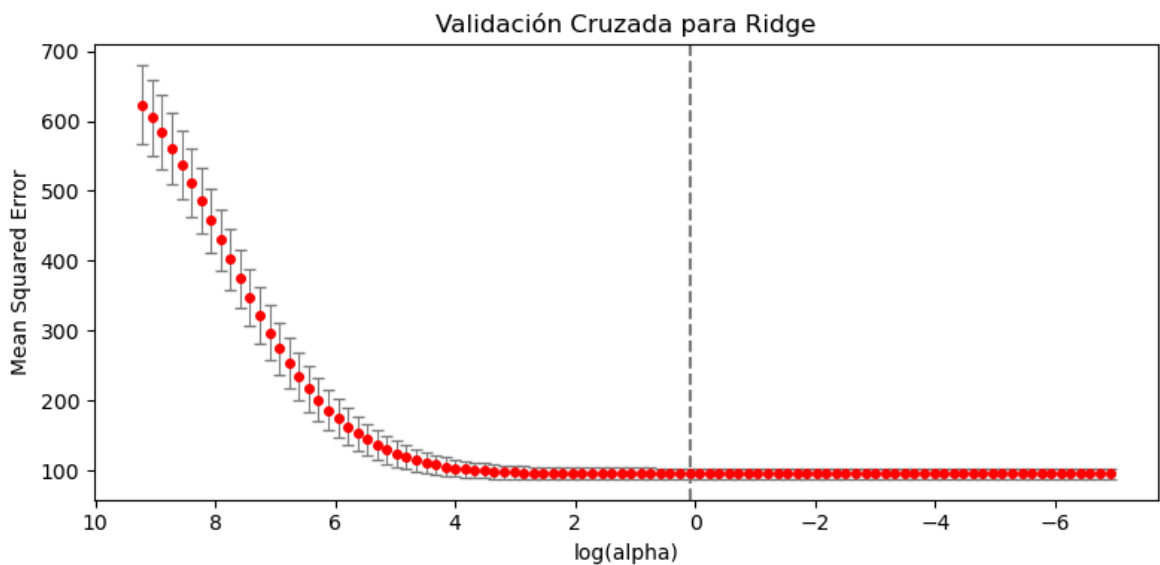
```
#Realizar el gráfico
x = np.log(alphas)

plt.figure(figsize=(8, 4))
plt.errorbar(x, mse_mean, yerr=mse_std, fmt='o', color='red', ecolor='gray', eli
plt.axvline(np.log(alpha_min), linestyle='--', color="gray")

plt.xlabel("log(alpha)")
plt.ylabel('Mean Squared Error')
plt.title('Validación Cruzada para Ridge')
plt.gca().invert_xaxis()

plt.tight_layout()
plt.show()

print("El valor de alpha que minimiza el mse es %s" %alpha_min_ridge)
```



El valor de alpha que minimiza el mse es 1.0974987654930557

Parte 2

Realizar lo siguiente:

- Realizar un modelo de regresión Lasso del nivel de productividad con $\alpha=0.05$ y $\alpha=1$
- Comparar los dos modelos anterior con la regresión lineal múltiple.
- Realizar la validación cruzada para determinar el valor de α que proporciona un menor error cuadrático

In [127...

```
#Importar clase
from sklearn.linear_model import Lasso

#Ajustar el modelo de Lasso con alfa=0.05
lasso_reg = Lasso(alpha = 0.05)
lasso_reg.fit(X_train, y_train)

#Intercepto
print("El intercepto que se tiene es %s" %lasso_reg.intercept_)

#Coeficientes de regresion
```

```
coefs_lasso_a = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":  
coefs_lasso_a
```

El intercepto que se tiene es 37.66410942195378

Out[127...

	Variables	Coeficientes
0	age	0.155112
1	daily_screen_time	0.000000
2	social_media_hours	-17.417912
3	study_hours	22.574835
4	sleep_hours	7.636898
5	notifications_per_day	-2.743889
6	focus_score	-5.465654

In [130...

```
#Importar clase
from sklearn.linear_model import Lasso

#Ajustar el modelo de Lasso con alfa=1
lasso_reg_b = Lasso(alpha = 1)
lasso_reg_b.fit(X_train, y_train)

#Intercepto
print("El intercepto que se tiene es %s" %lasso_reg_b.intercept_)
#Coeficientes de regresion
coefs_lasso_b = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":  
coefs_lasso_b
```

El intercepto que se tiene es 37.5142158682236

Out[130...

	Variables	Coeficientes
0	age	0.000000
1	daily_screen_time	-0.000000
2	social_media_hours	-13.279101
3	study_hours	19.528106
4	sleep_hours	5.410217
5	notifications_per_day	-0.780235
6	focus_score	-0.000000

In [132...

```
fig = plt.figure(figsize=(10, 6))

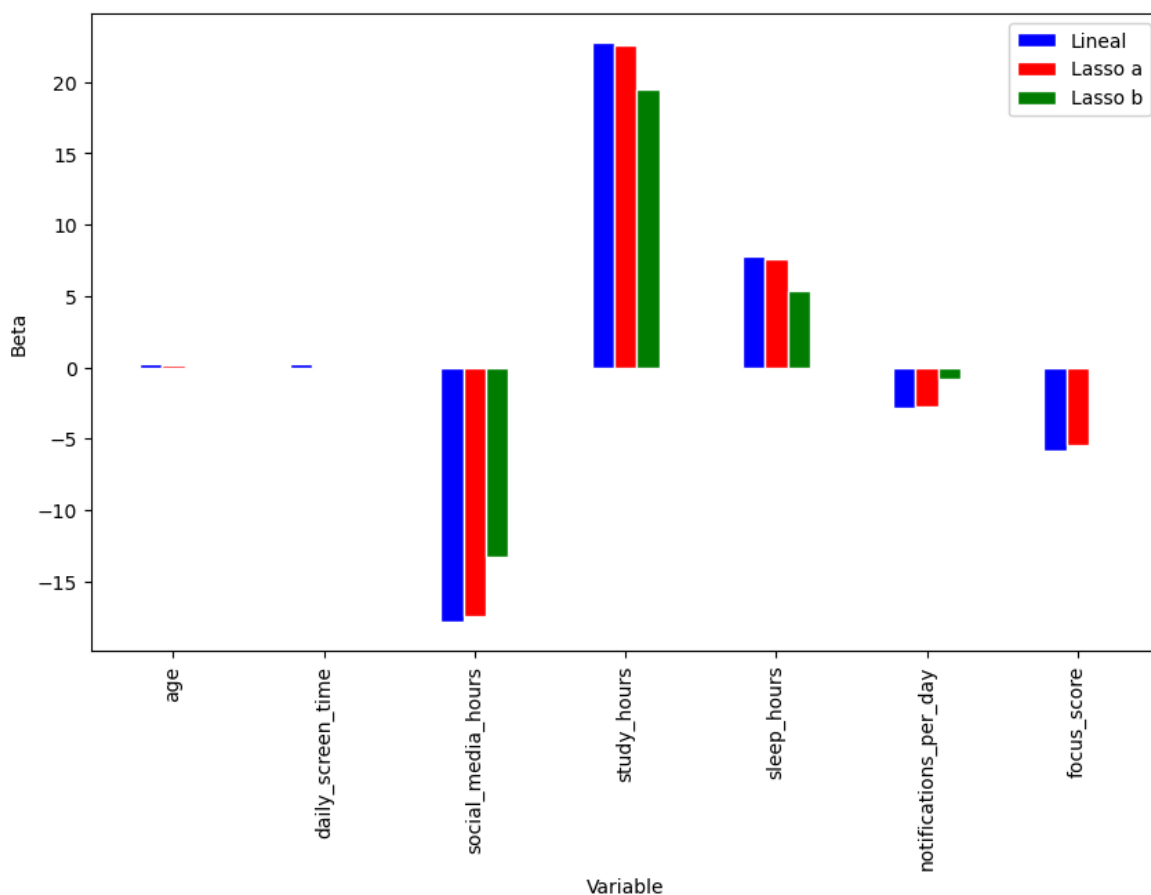
#Ancho de barra
barWidth = 0.15

#Definir posicion barras series
r1 = np.arange(len(lm.coef_))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]

#Pintar las barras
```

```
plt.bar(r1, lm.coef_, color = "blue", width = barWidth, edgecolor = "white", label=r1)
plt.bar(r2, lasso_reg.coef_, color = "red", width = barWidth, edgecolor = "white", label=r2)
plt.bar(r3, lasso_reg_b.coef_, color = "green", width = barWidth, edgecolor = "white", label=r3)

plt.xticks([r + barWidth for r in range(len(lm.coef_))],
            [predictoras.columns[i] for i in range(len(predictoras.columns))], rotate=45)
plt.legend()
plt.xlabel("Variable");
plt.ylabel("Beta");
plt.show(fig)
```



In [141]...

```
#Obtener error para cada uno
y_pred_lasso = lasso_reg.predict(X_test)
y_pred_lasso_b = lasso_reg_b.predict(X_test)

mse_lasso = mean_squared_error(y_test, y_pred_lasso)
mse_lasso_b = mean_squared_error(y_test, y_pred_lasso_b)
errores_lasso = [mse_lm, mse_lasso, mse_lasso_b]

comparación_lasso = pd.DataFrame({"Modelo": ["Lineal", r"Lasso $\alpha=0.05$", r"Lasso $\alpha=1$"],
                                  "MSE": errores_lasso})

comparación_lasso
```

Out[141]...

	Modelo	MSE
0	Lineal	96.709343
1	Lasso $\alpha=0.05$	96.525596
2	Lasso $\alpha=1$	111.830536

```
In [75]: from sklearn.linear_model import LassoCV, lasso_path

# Ajustar LassoCV
lasso_cv = LassoCV(cv=10, random_state=2026, n_alphas=100)
lasso_cv.fit(X_train, y_train)

alphas = lasso_cv.alphas_
mse_path = lasso_cv.mse_path_
mse_mean = mse_path.mean(axis=1)
mse_std = mse_path.std(axis=1)
alpha_min = lasso_cv.alpha_

idx_min = np.argmin(mse_mean)
mse_min = mse_mean[idx_min]
se_min = mse_std[idx_min]
idx_1se = np.where(mse_mean <= mse_min + se_min)[0][0]
alpha_1se = alphas[idx_1se]

#scaler = StandardScaler()
#X_scaled = scaler.fit_transform(predictoras_est)
alphas_path, coefs_path, _ = lasso_path(X_train, y_train, alphas=alphas)
n_nonzero = np.sum(coefs_path != 0, axis=0)

x = np.log(alphas)

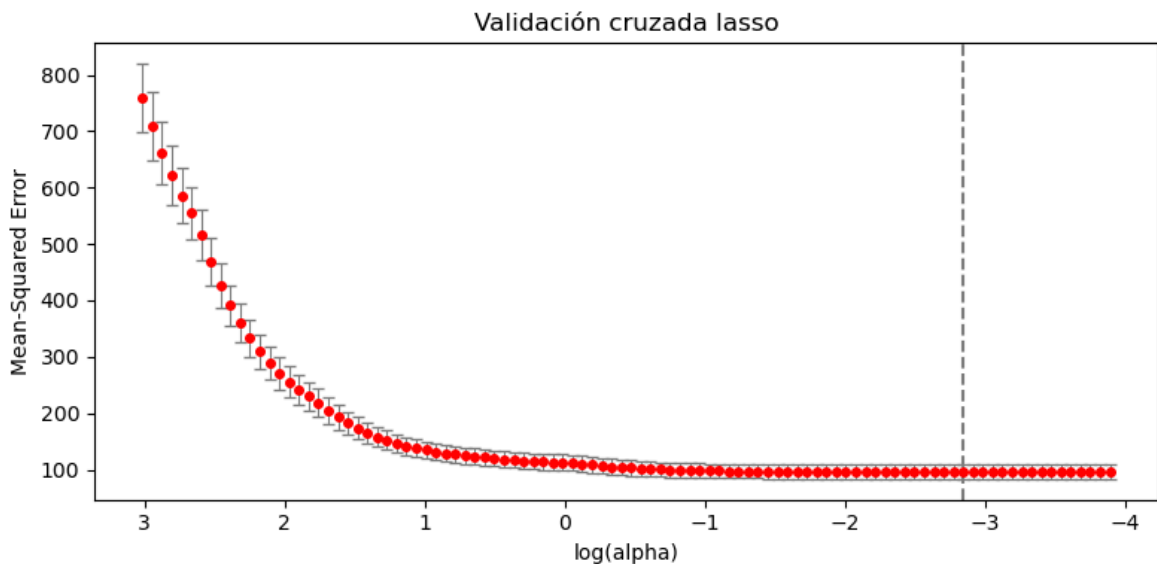
fig, ax = plt.subplots(figsize=(8, 4))

plt.errorbar(x, mse_mean, yerr=mse_std, fmt='o', color='red', ecol='gray', eli

ax.set_xlabel('log(alpha)')
ax.set_ylabel('Mean-Squared Error')
ax.set_title('Validación cruzada lasso')
ax.invert_xaxis()

plt.axvline(np.log(alpha_min), linestyle='--', color="gray")
plt.tight_layout()
plt.show()

print("El valor de alpha que minimiza el mse es %s" %alpha_min )
```



El valor de alpha que minimiza el mse es 0.05801500605329416

Parte 3

Realizar un modelo de Red elástica del nivel de productividad con las variables restantes.

- Fijando $r=0.1$
- Fijando $r=0.90$ (En ambos casos ajustar por validación cruzada con $k=10$ particiones)
- Comparar los modelos con el modelo regresión lineal múltiple. ¿Qué se observa?
- Realizar la validación cruzada para determinar el valor de α y r que proporciona un menor error cuadrático.

```
In [46]: #Importar clase
from sklearn.linear_model import ElasticNetCV

#Ajustar el modelo con r=0.1
e_net = ElasticNetCV(cv = 10, l1_ratio = 0.1, random_state=2026)
e_net.fit(X_train, y_train)

#Obtener coeficientes del modelo

#Intercepto
print("El intercepto que se tiene es %s" %e_net.intercept_)
#Coeficientes de regresion
coefs_e_net = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes": e_
coefs_e_net
```

El intercepto que se tiene es 37.631829847138825

```
Out[46]:
```

	Variables	Coeficientes
0	age	-0.033447
1	daily_screen_time	-2.812464
2	social_media_hours	-9.790830
3	study_hours	17.160263
4	sleep_hours	5.307172
5	notifications_per_day	-1.322846
6	focus_score	0.689663

```
In [49]: #Importar clase
from sklearn.linear_model import ElasticNetCV

#Ajustar el modelo con r=0.9
e_net_b = ElasticNetCV(cv = 10, l1_ratio = 0.9, random_state=2026)
e_net_b.fit(X_train, y_train)

#Obtener coeficientes del modelo

#Intercepto
```

```
print("El intercepto que se tiene es %s" %e_net.intercept_)
#Coeficientes de regresion
coefs_e_net_b = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":
coefs_e_net_b
```

El intercepto que se tiene es 37.631829847138825

Out[49]:

	Variables	Coeficientes
0	age	0.184403
1	daily_screen_time	0.039622
2	social_media_hours	-17.462721
3	study_hours	22.566544
4	sleep_hours	7.656887
5	notifications_per_day	-2.772168
6	focus_score	-5.498530

In [61]:

```
#Obtener error para cada uno
y_pred_e_net = e_net.predict(X_test)
y_pred_e_net_b = e_net_b.predict(X_test)

mse_e_net = mean_squared_error(y_test, y_pred_e_net)
mse_e_net_b = mean_squared_error(y_test, y_pred_e_net_b)

errores_e_net = [mse_lm,mse_e_net, mse_e_net_b]

comparación_e_net= pd.DataFrame({"Modelo": ["Lineal", r"Red elástica $r=0.1$", r
"MSE": errores_e_net})

comparación_e_net
```

Out[61]:

	Modelo	MSE
0	Lineal	96.709343
1	Red elástica \$r=0.1\$	124.401374
2	Red elástica \$r=0.9\$	96.545072

In [68]:

```
fig = plt.figure(figsize=(9, 6))

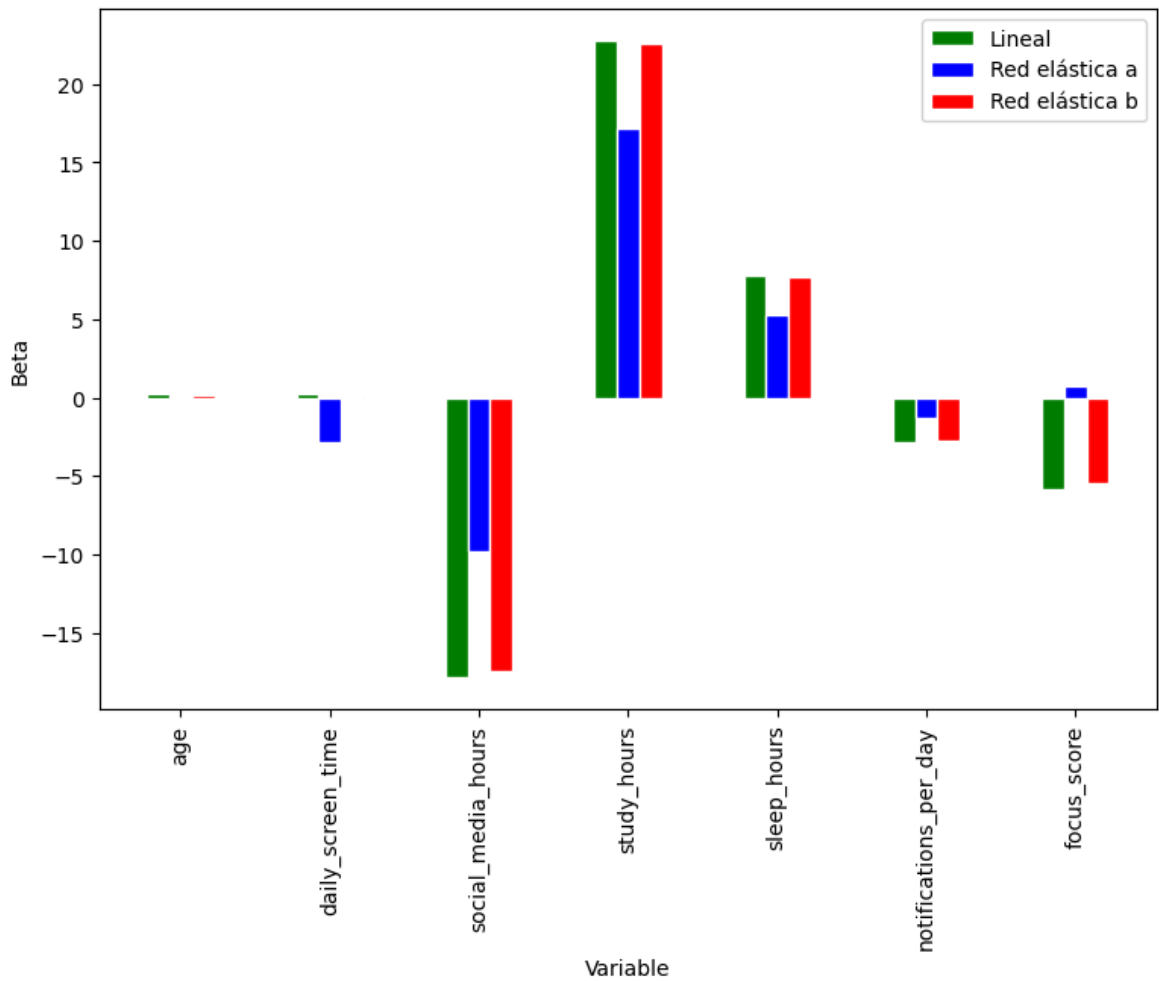
#Ancho de barra
barWidth = 0.15

#Definir posicion barras series
r1 = np.arange(len(lm.coef_))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]

#Pintar las barras
plt.bar(r1, lm.coef_, color = "green", width = barWidth, edgecolor = "white", la
plt.bar(r2, e_net.coef_, color = "blue", width = barWidth, edgecolor = "white",
plt.bar(r3, e_net_b.coef_, color = "red", width = barWidth, edgecolor = "white",

plt.xticks([r + barWidth for r in range(len(lm.coef_))],
```

```
[predictoras.columns[i] for i in range(len(predictoras.columns))], rotation = 90
plt.legend();
plt.xlabel("Variable");
plt.ylabel("Beta");
plt.show(fig)
```



```
In [79]: from sklearn.linear_model import ElasticNet, ElasticNetCV
from sklearn.model_selection import KFold, cross_val_score

modelo_cv = ElasticNetCV(alphas=alphas, l1_ratio=np.linspace(0.1, 0.99, 10),
cv=10, max_iter=1000, random_state=2026)

modelo_cv.fit(X_train, y_train)

print("El valor de alfa que minimiza el error es %s" %modelo_cv.alpha_)
print("El valor de r que minimiza el error es %s" %modelo_cv.l1_ratio_)
```

El valor de alfa que minimiza el error es 0.0541049888147524

El valor de r que minimiza el error es 0.99

In []: